CSCI 112 Assignment 8 – File Access and Math Functions

Points: 10

Automated tests will run on GitHub Actions after every push. Check the Actions tab to see which tests pass and which fail.

Instructions

1. **Fork** the repository on GitHub - `https://github.com/jpach-cs/112-ass08-template`.
2. Clone your forked repository to your local machine.
3. Open the file `code.h` and add:
    1. Include guards at the top: `#ifndef CODE_H #define CODE_H`
    2. Required includes (`<stdbool.h>`)
    3. All function prototypes
    4. Include guards at the bottom: `#endif`
4. Open the file `code.c`.
    1. Include `<stdio.h>`, `<stdlib.h>` (for `abs()`), `<math.h>` (for the other math functions) and `code.h`.
    2. Implement every function listed below:
        - Read the description of each function carefully.
        - Follow the rules — use ONLY the standard library (`stdio.h`, `stdlib.h`, `math.h`, `stdbool.h`).
        - Use the **exact file modes** and **exact file formats** specified. The tests inspect your files byte by byte.
    3. Test your implementation locally using `main.c`.
5. Once all functions are complete — save your changes, then **commit** and **push** to your repository.
6. Go to the **Actions** tab on GitHub and check the test results.

Learning Objectives

After completing this lab, you will:

- Create, read, append to and clear files in **text mode** (`"w"`, `"r"`, `"a"`)
- Create, read, append to and clear files in **binary mode** (`"wb"`, `"rb"`, `"ab"`)
- Write and read text with `fprintf()` / `fscanf()`
- Write and read raw bytes with `fwrite()` / `fread()`
- Navigate a binary file with `fseek()` and `ftell()`
- Handle errors (`fopen()` returning `NULL`)
- Use the math functions: `abs()`, `fabs()`, `sqrt()`, `pow()`, `ceil()`, `floor()`, `round()`, `fmod()`

General rules

1. Every function that returns `bool` returns `true` on success and `false` on any failure (`fopen()` failed, invalid argument, read/write error).
2. **Always check** the result of `fopen()`.
3. **Always close** every file you open (also on error paths after a successful `fopen()`).
4. If an argument is invalid (see each function), return `false` **without touching the file** (do not create, truncate or modify it).
5. All floating point numbers in **text** files are written with exactly two digits after the decimal point: `"%.2f\n"`. Integers are written with `"%d\n"`. Every value is on its own line and every line ends with `\n`.
6. In **binary** files an `int` is stored as `sizeof(int)` bytes and a `double` as `sizeof(double)` bytes, with no separators and no extra data.

Implement 20 functions

Part A – Text mode

#	Function	Mode	Math	Description
1	`bool create_abs_txt(const char *filename, int value)`	`"w"`	`abs()`	Creates (or overwrites) the file. Content: `abs(value)` as text.
2	`bool create_fabs_txt(const char *filename, double value)`	`"w"`	`fabs()`	Creates (or overwrites) the file. Content: `fabs(value)` as text.
3	`bool append_sqrt_txt(const char *filename, double value)`	`"a"`	`sqrt()`	Appends `sqrt(value)` as a new line. Invalid: `value < 0`.
4	`bool append_pow_txt(const char *filename, double base, double exponent)`	`"a"`	`pow()`	Appends `pow(base, exponent)` as a new line. Invalid: `base == 0` and `exponent < 0`.

#	Function	Mode	Math	Description
5	`bool create_ceil_floor_txt(const char *filename, double value)`	`"w"`	`ceil()`, `floor()`	Creates (or overwrites) the file with **two lines**: line 1 = `ceil(value)`, line 2 = `floor(value)`.
6	`bool append_round_txt(const char *filename, double value)`	`"a"`	`round()`	Appends `round(value)` as a new line.
7	`bool append_fmod_txt(const char *filename, double x, double y)`	`"a"`	`fmod()`	Appends `fmod(x, y)` as a new line. Invalid: `y == 0`.
8	`int load_txt_values(const char *filename, double *arr, int max_count)`	`"r"`	–	Reads numbers from a text file with `fscanf()` into `arr`, at most `max_count` values. Returns the number of values read, or `-1` if the file cannot be opened.
9	`bool clear_file(const char *filename)`	`"w"`	–	Removes all content of the file, leaving an empty file (size 0). Creates it if it does not exist.

Text file examples

```
create_abs_txt("a.txt", -7)            -> a.txt contains:  7\n
create_fabs_txt("a.txt", -2.5)         -> a.txt contains:  2.50\n
append_sqrt_txt("a.txt", 16.0)         -> adds the line:   4.00\n
append_pow_txt("a.txt", 2.0, 10.0)     -> adds the line:   1024.00\n
create_ceil_floor_txt("a.txt", 2.3)    -> a.txt contains:  3.00\n2.00\n
append_round_txt("a.txt", 2.5)         -> adds the line:   3.00\n
append_fmod_txt("a.txt", 7.5, 2.0)     -> adds the line:   1.50\n
```

Part B – Binary mode

#	Function	Mode	Math	Description
10	`bool create_abs_bin(const char *filename, int value)`	`"wb"`	`abs()`	Creates (or overwrites) the file. Content: **one `int`** = `abs(value)`.
11	`bool create_fabs_bin(const char *filename, double value)`	`"wb"`	`fabs()`	Creates (or overwrites) the file. Content: **one `double`** = `fabs(value)`.
12	`bool append_sqrt_bin(const char *filename, double value)`	`"ab"`	`sqrt()`	Appends one `double` = `sqrt(value)`. Invalid: `value < 0`.
13	`bool append_pow_bin(const char *filename, double base, double exponent)`	`"ab"`	`pow()`	Appends one `double` = `pow(base, exponent)`. Invalid: `base == 0` and `exponent < 0`.
14	`bool create_ceil_floor_bin(const char *filename, double value)`	`"wb"`	`ceil()`, `floor()`	Creates (or overwrites) the file with **two `doubles`**: first `ceil(value)`, then `floor(value)`.
15	`bool append_round_bin(const char *filename, double value)`	`"ab"`	`round()`	Appends one `double` = `round(value)`.
16	`bool append_fmod_bin(const char *filename, double x, double y)`	`"ab"`	`fmod()`	Appends one `double` = `fmod(x, y)`. Invalid: `y == 0`.
17	`bool load_abs_bin(const char *filename, int *out)`	`"rb"`	–	Reads the first `int` from the file into `*out`. Returns `false` if the file cannot be opened or does not contain a full `int`.

#	Function	Mode	Math	Description
18	`int load_bin_values(const char *filename, double *arr, int max_count)`	`"rb"`	–	Reads `double`s into `arr`, at most `max_count`. Returns the number of values read, or `-1` if the file cannot be opened.
19	`bool load_bin_value_at(const char *filename, int index, double *out)`	`"rb"`	–	Reads the `double` stored at position `index` (0-based) using `fseek()`. Returns `false` if the file cannot be opened, `index < 0`, or the file is too short.
20	`long get_file_size(const char *filename)`	`"rb"`	–	Returns the size of the file in bytes using `fseek()` and `ftell()`. Returns `-1` if the file cannot be opened.

Binary file examples

```
create_ceil_floor_bin("b.bin", 2.3)   -> b.bin: [3.0 as double][2.0 as double]   (2 * sizeof
append_sqrt_bin("b.bin", 9.0)         -> b.bin: [3.0][2.0][3.0]                   (3 * sizeof
load_bin_value_at("b.bin", 1, &v)     -> v == 2.0
get_file_size("b.bin")                -> 3 * sizeof(double)
```

Background

Opening and closing a file

```c
FILE *f = fopen("data.txt", "w");
if (f == NULL)
{
    return false;       // always check!
}
fprintf(f, "%d\n", 42);
fclose(f);              // always close!
return true;
```

Modes you need

Mode	Meaning
`"w"` / `"wb"`	Write. Creates the file, **erases** existing content.
`"a"` / `"ab"`	Append. Creates the file if needed, adds data at the **end**.
`"r"` / `"rb"`	Read. The file must exist.

Text vs. binary

```c
// text: human readable
fprintf(f, "%.2f\n", 3.5);          // writes the characters 3 . 5 0 \n
fscanf(f, "%lf", &x);               // reads a double (note: %lf for double)

// binary: raw bytes
double d = 3.5;
fwrite(&d, sizeof(d), 1, f);        // writes sizeof(double) bytes
fread(&d, sizeof(d), 1, f);         // returns the number of elements read
```

Moving around a binary file

```c
fseek(f, index * sizeof(double), SEEK_SET);   // jump to the index-th double
fseek(f, 0, SEEK_END);                        // jump to the end
long size = ftell(f);                         // current position = size in bytes
```

Math functions

Function	Header	Meaning	Example
`int abs(int)`	`stdlib.h`	absolute value of an integer	`abs(-7)` → `7`
`double fabs(double)`	`math.h`	absolute value of a double	`fabs(-2.5)` → `2.5`
`double sqrt(double)`	`math.h`	square root	`sqrt(16.0)` → `4.0`
`double pow(double, double)`	`math.h`	power	`pow(2.0, 10.0)` → `1024.0`
`double ceil(double)`	`math.h`	round **up**	`ceil(2.3)` → `3.0`
`double floor(double)`	`math.h`	round **down**	`floor(2.7)` → `2.0`
`double round(double)`	`math.h`	round to nearest (halves away from zero)	`round(2.5)` → `3.0`

Function	Header	Meaning	Example
`double fmod(double, double)`	`math.h`	floating point remainder	`fmod(7.5, 2.0)` → `1.5`

Note: when compiling by hand, programs using `math.h` functions may need the `-lm` flag: `gcc main.c code.c -o main -lm`

Only for testing

Local Testing (Sandbox)

Create test cases in main.c:

```c
#include <stdio.h>
#include "code.h"

int main()
{
    // Text mode
    create_abs_txt("test.txt", -7);
    append_sqrt_txt("test.txt", 16.0);
    append_pow_txt("test.txt", 2.0, 10.0);
    append_round_txt("test.txt", 2.5);
    append_fmod_txt("test.txt", 7.5, 2.0);

    double values[10];
    int n = load_txt_values("test.txt", values, 10);
    printf("Read %d values from text file:\n", n);
    for (int i = 0; i < n; i++)
    {
        printf("  %f\n", values[i]);
    }

    clear_file("test.txt");
    printf("Size after clear: %ld\n", get_file_size("test.txt"));

    // Binary mode
    create_ceil_floor_bin("test.bin", 2.3);
    append_sqrt_bin("test.bin", 9.0);
    append_pow_bin("test.bin", 3.0, 2.0);

    n = load_bin_values("test.bin", values, 10);
```

```
    printf("Read %d values from binary file:\n", n);
    for (int i = 0; i < n; i++)
    {
        printf("  %f\n", values[i]);
    }

    double v;
    if (load_bin_value_at("test.bin", 2, &v))
    {
        printf("Value at index 2: %f\n", v);
    }
    printf("Binary file size: %ld bytes\n", get_file_size("test.bin"));

    create_abs_bin("abs.bin", -42);
    int iv;
    if (load_abs_bin("abs.bin", &iv))
    {
        printf("abs.bin: %d\n", iv);
    }

    return 0;
}
```